

MNC Logistics Report

Technical Guide

Balloon landings, airstrip operations, airstrip maintenance, and airline complaints reporting

Generated August 08, 2026

Workflow id: **logistics_report**

1. Overview

The **logistics_report** workflow (repository: `mnc_logistics_report`) fetches events from EarthRanger for a specified time window — specifically **balloon_landing**, **airstrip_operations**, **airstrip_maintenance**, and **airline_complaint** event types — and routes them into four independent logistics reporting branches. Each branch processes and flattens event details, cleans and renames fields, and persists its data as a CSV table. The three published branches additionally render an interactive HTML table widget that is attached to the workflow's dashboard.

The workflow delivers:

- **balloon_landing_summary_table.csv** — passenger records per balloon company and lodge
- **airstrip_operations_summary_table.csv** — total client counts pivoted by camp/lodge and direction (arrival / departure)
- **airstrip_maintenance_summary_table.csv** — dated log of airstrip maintenance activity types
- **MNC Logistics Dashboard** — a dashboard populated with one sortable/filterable table widget per summary table above

Output summary

Output file	Source event type	Description
balloon_landing_summary_table.csv	balloon_landing	Passenger records: Date, Balloon Company, Where Are Clients Staying, No Of Passengers
airstrip_operations_summary_table.csv	airstrip_operations	Pivoted summary: Camp Lodge × Arrival/Departure, total clients
airstrip_maintenance_summary_table.csv	airstrip_maintenance	Dated activity log: Date, Maintenance Type

2. Dependencies

2.1 Python packages

Package	Version	Channel
ecoscope-platform	>=2.15.0, <2.16.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge

Note: **ecoscope-platform** replaces the previously separate **ecoscope-workflows-core** and **ecoscope-workflows-ext-ecoscope** packages. The **ecoscope-workflows-ext-mep** and **ecoscope-workflows-ext-big-life** packages, previously listed, are no longer required by this workflow. **opentelemetry-sdk** enables distributed tracing for the compiled workflow's task graph; **pydeck** is a platform-level requirement pulled in alongside **ecoscope-platform**.

2.2 Connection

Connection	Task	Purpose
EarthRanger	set_er_connection	Fetch event records (balloon_landing, airstrip_operations, airstrip_maintenance, airline_complaint) for the analysis time range. The client is also passed to each process_events_details call to resolve display titles for event detail fields.

Note: This workflow does not require Google Earth Engine or any Dropbox file downloads.

2.3 Grouper

The workflow uses an **empty grouper list** (groupers: []). All event records are processed as a single undivided dataset — no fan-out or per-group branching is applied to the data. The grouper is passed through to the dashboard only.

3. Event Ingestion Pipeline

All four reporting branches share a common ingestion pipeline that fetches, date-stamps, and temporally indexes events before branching.

3.1 Event retrieval

Parameter	Value
Task	get_events
event_types	balloon_landing, airstrip_operations, airstrip_maintenance, airline_complaint
Columns retained	id, time, event_type, event_category, reported_by, serial_number, geometry, created_at, event_details, patrols
include_details	true
raise_on_empty	true
include_null_geometry	false
include_updates	false
include_related_events	false
include_display_values	false
force_point_geometry	true

Note: Only the four event types required by this workflow are fetched at retrieval time. Downstream filter_df steps then isolate each type into its own branch. force_point_geometry: true normalises all event geometries to points before they reach the branch pipelines.

3.2 Date extraction and temporal indexing

Step	Task	Detail
1	extract_column_as_type	Extract the time column as output_type: date into a new column named date . This strips the time component for daily grouping downstream.
2	add_temporal_index	Add temporal index using time_col: date , groupers: [], cast_to_datetime: true, format: mixed. Produces the shared events_temporal DataFrame used by all four branches.

3.3 Common branch pattern

Every branch follows the same four-step normalisation pattern before any branch-specific transformations:

Step	Task	Detail
1	filter_df	Filter events_temporal by event_type (op: equal, reset_index: false).

Step	Task	Detail
2	process_events_details	Resolve event detail field IDs to their display titles (map_to_titles: true, ordered: true). Requires the EarthRanger client to look up schema definitions.
3	normalize_json_column	Flatten the event_details JSON column into individual columns (skip_if_not_exists: true, sort_columns: true).
4	drop_column_prefix	Remove the event_details__ prefix from all flattened columns (duplicate_strategy: keep_original).

Note: Because map_to_titles is true, flattened column names reflect human-readable field titles (e.g. 'Balloon Company', 'Camp/Lodge') rather than raw field keys. map_columns steps reference these titles directly.

4. Branch 1 — Balloon Landings

Filters **balloon_landing** events and produces a passenger summary table grouped by balloon company and lodge.

4.1 Normalisation

Steps 1–4 follow the common branch pattern described in Section 3.3.

4.2 Column selection and renaming

Task: **map_columns** (raise_if_not_found: false). Only the following columns are retained after prefix removal:

Source column (title after prefix drop)	Renamed to
date	date (retained, not renamed)
Balloon Company	balloon_company
Where are clients staying?	where_are_clients_staying
# of passengers	no_of_passengers

Note: All other columns are discarded at this step. raise_if_not_found: false means the task skips gracefully if any of the named columns are absent.

4.3 Cleaning, display renaming, persistence, and widget

Step	Task	Detail
1	ecoscope_workflows_ext_mnc.tasks.transformation.remove_brackets_from_column	Strip bracket characters from balloon_company and where_are_clients_staying columns.
2	map_columns	Rename columns to display-friendly headers: date→Date, balloon_company→Balloon Company, where_are_clients_staying→Where Are Clients Staying, no_of_passengers→No Of Passengers.
3	persist_df	Save as balloon_landing_summary_table.csv (filetype: csv), using the display-renamed table.
4	draw_table	Render the display-renamed table as an HTML widget (widget_id: "Balloon Landing Summary"; sorting and filtering enabled; download disabled).
5	persist_text	Save the rendered HTML as a text file (filename_suffix: balloon_landing_summary_table.html).
6	create_table_widget_single_view	Wrap the persisted HTML into a dashboard widget titled "Balloon Landing Summary", referenced by the dashboard's widgets list.

5. Branch 2 — Airstrip Operations

Filters **airstrip_operations** events and produces a pivoted summary of total client counts by camp/lodge and direction (arrival or departure).

5.1 Normalisation

Steps 1–4 follow the common branch pattern described in Section 3.3.

5.2 Column renaming

Task: **map_columns** (raise_if_not_found: false, retain_columns: []). No columns are dropped; all are retained with the following renames:

Source column (title after prefix drop)	Renamed to
Airline	airline
Arrival or departure	arrival_departure
Attendant	attendant
Camp/Lodge	camp_lodge
Number of clients	no_of_clients

5.3 Cleaning

The previous null-replacement (`replace_empty_strings_in_columns`) and `camp_lodge` capitalisation (`format_text_column`) steps have been removed from this branch; cleaning now goes directly from bracket-stripping to integer coercion.

Step	Task	Detail
1	<code>ecoscope_workflows_ext_mnc.tasks.transformation.remove_brackets_from_column</code>	Strip bracket characters from airline , arrival_departure , attendant , and camp_lodge .
2	<code>ecoscope_workflows_ext_mnc.tasks.transformation.convert_columns_to_int</code>	Cast no_of_clients to integer (errors: coerce, fill_value: 0).

5.4 Summary, pivot, display renaming, persistence, and widget

Step	Task	Detail
1	<code>summarize_df</code>	Group by [camp_lodge , arrival_departure] and compute sum(no_of_clients) (display_name: no_of_clients, decimal_places: 0). <code>reset_index: true</code> .
2	<code>pivot_dataframe</code>	Pivot the summary table: index: camp_lodge , columns: arrival_departure , values: no_of_clients , fill_value: 0. Produces one column per direction value (arrival, departure).

Step	Task	Detail
3	<code>ecoscope_workflows_ext_mnc.tasks.transformation.convert_columns_to_int</code>	Cast pivot columns arrival and departure to integer (errors: coerce, fill_value: 0).
4	<code>map_columns</code>	Rename columns to display-friendly headers: <code>camp_lodge</code> →Camp Lodge, <code>no_of_clients_Arrival</code> →Arrival, <code>no_of_clients_Departure</code> →Departure.
5	<code>persist_df</code>	Save as <code>airstrip_operations_summary_table.csv</code> (filetype: csv), using the display-renamed table.
6	<code>draw_table</code>	Render the display-renamed table as an HTML widget (widget_id: "Airstrip Operations Summary"; sorting and filtering enabled; download disabled).
7	<code>persist_text</code>	Save the rendered HTML as a text file (filename_suffix: <code>airstrip_operations_summary_table.html</code>).
8	<code>create_table_widget_single_view</code>	Wrap the persisted HTML into a dashboard widget titled "Airstrip Operations Summary", referenced by the dashboard's widgets list.

6. Branch 3 — Airstrip Maintenance

Filters **airstrip_maintenance** events and produces a dated log of maintenance activity types.

6.1 Normalisation

Steps 1–4 follow the common branch pattern described in Section 3.3.

6.2 Column selection and renaming

Task: **map_columns** (raise_if_not_found: false). Only the following columns are retained:

Source column (title after prefix drop)	Renamed to
date	date (retained, not renamed)
Maintenance type	maintenance_type

Note: All other event detail columns are discarded at this step.

6.3 Display renaming, persistence, and widget

Step	Task	Detail
1	map_columns	Rename columns to display-friendly headers: date→Date, maintenance_type→Maintenance Type.
2	persist_df	Save as airstrip_maintenance_summary_table.csv (filetype: csv), using the display-renamed table.
3	draw_table	Render the display-renamed table as an HTML widget (widget_id: “Airstrip Maintenance Summary”; sorting and filtering enabled; download disabled).
4	persist_text	Save the rendered HTML as a text file (filename_suffix: airstrip_maintenance_summary_table.html).
5	create_table_widget_single_view	Wrap the persisted HTML into a dashboard widget titled “Airstrip Maintenance Summary”, referenced by the dashboard's widgets list.

7. Branch 4 — Airline Complaints

Filters **airline_complaint** events and normalises their event details using the same four-step common branch pattern.

7.1 Normalisation

Step	Task	Detail
1	filter_df	Filter events_temporal to rows where event_type == 'airline_complaint' (op: equal, reset_index: false).
2	process_events_details	Resolve event detail field IDs to display titles (map_to_titles: true, ordered: true).
3	normalize_json_column	Flatten the event_details JSON column (skip_if_not_exists: true, sort_columns: true).
4	drop_column_prefix	Remove the event_details__ prefix (duplicate_strategy: keep_original). This is the terminal step in the current spec for this branch.

Note: The airline complaint branch currently ends at the drop_column_prefix step. Column selection, renaming, persistence, and widget steps are not yet defined in the spec. Like every other task in the workflow, all four steps in this branch inherit the global skipif default (any_is_empty_df, any_dependency_skipped) — see Section 9.1.

8. Output Files

All outputs are written to **ECOSCOPE_WORKFLOWS_RESULTS**.

File	Branch	Columns	Description
balloon_landing_summary_table.csv	Balloon landings	Date, Balloon Company, Where Are Clients Staying, No Of Passengers	Passenger records by balloon company and lodge
airstrip_operations_summary_table.csv	Airstrip operations	Camp Lodge, Arrival, Departure (pivoted)	Total clients per camp/lodge pivoted by direction
airstrip_maintenance_summary_table.csv	Airstrip maintenance	Date, Maintenance Type	Dated log of airstrip maintenance activity types

8.1 Dashboard widget HTML

Each of the three CSV outputs above also has a matching rendered HTML table persisted (via `draw_table → persist_text`) as `<filename>_summary_table.html`. These HTML files are the data source referenced by the corresponding `create_table_widget_single_view` widget in the MNC Logistics Dashboard — they are not intended to be opened directly, but are addressed by the dashboard's widgets list.

9. Workflow Execution Logic

9.1 Global skip conditions

This workflow now defines a single **task-instance-defaults** block at the top of the spec, which applies the same skipif conditions to every task automatically. Individual tasks no longer repeat their own skipif block:

Condition	Behaviour
any_is_empty_df	Skip this task if any input DataFrame is empty
any_dependency_skipped	Skip this task if any upstream dependency was skipped

Note: This is a behaviour-preserving simplification over the previous spec, which declared the identical skipif block on every task individually. Because the conditions are unchanged, each of the branches still propagates skips independently — if balloon_landing events are absent, only that branch (and its widget) is skipped; the other branches continue normally.

9.2 Four branches, three published to the dashboard

After the shared ingestion pipeline produces **events_temporal**, the workflow splits into four independent branches. Each branch reads from events_temporal and produces its own output with no cross-branch dependencies:

Branch	Filter value	Output	Dashboard widget
Balloon landings	balloon_landing	balloon_landing_summary_table.csv	Balloon Landing Summary
Airstrip operations	airstrip_operations	airstrip_operations_summary_table.csv	Airstrip Operations Summary
Airstrip maintenance	airstrip_maintenance	airstrip_maintenance_summary_table.csv	Airstrip Maintenance Summary
Airline complaints	airline_complaint	(normalised; no persist step yet)	— (none)

9.3 No mapvalues or fan-out

This workflow processes all records as a single batch. There is no **mapvalues**, **split_groups**, or **zip_groupbykey** — every task runs exactly once.

9.4 Table widgets, but no charts or maps

This workflow produces no charts, maps, or map-based widgets — there are no html_to_png conversions, draw_map calls, or Likert/pie/bar chart tasks. It does, however, render three interactive HTML **table** widgets via draw_table (one per published branch), which are persisted as HTML text via persist_text and wrapped as dashboard widgets via create_table_widget_single_view.

9.5 Dashboard

The workflow concludes with **gather_dashboard** (id: logistics_dashboard, name: "MNC Logistics Dashboard"), which packages workflow details, time range, groupers, and the **widgets** list. The widgets list now references the three table widgets produced by the balloon landing, airstrip operations, and airstrip maintenance branches — previously this list was empty.

10. Software Versions

Package	Version pinned in spec.yml
ecoscope-platform	>=2.15.0, <2.16.0
ecoscope-workflows-ext-custom	0.1.0rc14.*
ecoscope-workflows-ext-ste	0.0.0rc1.*
ecoscope-workflows-ext-mnc	1.0.0.*
pydeck	0.9.2
opentelemetry-sdk	>=1.20.0, <2.0.0